

PRAKTEK TEKNOLOGI IOT II

LAPORAN PRAKTEK

TEKNOLOGI IOT II

Disusun dalam rangka memenuhi salah satu syarat kelengkapan nilai Praktikum

Mata Kuliah Teknologi IOT II



DISUSUN OLEH :

NAMA : Raffi Maulana Abdillah

NPM 25782021

PRODI : TEKNOLOGI REKAYASA INTERNET

KELAS : 3A

DOSEN : Dr. ir. Septafiansyah Dwi Putra, S. T.,M.T.

: Agus Ambarwari, S.Pd.,M.Kom.

MENTRI PENDIDIKAN DAN KEBUDAYAAN PROVINSI LAMPUNG

POLITEKNIK NEGERI LAMPUNG

TAHUN AJARAN 2025 / 2026

LATIHAN:

1. Pembahasan dan Analisis Praktikum

Sistem yang telah dibangun menggunakan arsitektur **Asynchronous Web Server** berbasis pustaka ESPAsyncWebServer dan protokol **WebSockets**.

- **Keunggulan Dibanding HTTP Standar:** Berbeda dengan modul sebelumnya yang menggunakan HTTP sinkron (request-response konvensional) di mana browser harus melakukan *refresh* atau menggunakan *meta refresh* (menyebabkan layar berkedip/flickering dan boros bandwidth), WebSockets membuka jalur pipa data persisten dua arah (*full-duplex*).
- **Mekanisme Push Data:** Ketika suhu berubah atau tombol fisik di breadboard ditekan, NodeMCU dapat secara proaktif mengirimkan (*push*) data berformat JSON ke browser tanpa menunggu permintaan dari klien. Hal ini membuat pembaruan antarmuka web menjadi instan dan mulus secara *real-time*.
- **Pemrograman Non-Blocking:** Penggunaan fungsi internal `millis()` menggantikan fungsi pemblokir seperti `delay()` memastikan mikrokontroler ESP8266 tetap siaga mengawasi interaksi jaringan dan tombol fisik tanpa mengalami *freeze*.

2. Jawaban Pertanyaan Latihan

Latihan 1 (Teori Jaringan)

Soal: Mengapa teknologi koneksi WebSockets jauh lebih hemat bandwidth dan tidak membuat layar berkedip dibandingkan metode Meta Refresh HTTP HTML?

- **Jawaban:** WebSockets menggunakan satu koneksi TCP yang terbuka secara terus-menerus (*persistent connection*). Setelah *handshake* awal berhasil, pertukaran data selanjutnya hanya mengirimkan paket data kecil berukuran beberapa byte (seperti format JSON) melalui pipa yang sama. Sebaliknya, metode Meta Refresh HTTP mengharuskan browser mengunduh ulang seluruh struktur halaman HTML, CSS, dan aset gambar dari awal setiap kali ada pembaruan. Hal inilah yang menyebabkan layar berkedip (*flickering*) serta membebani jaringan karena transfer data yang berulang-ulang dan berukuran besar.

Latihan 2 (Analisis Waktu/Timer)

Soal: Apa bahaya sistematis yang akan terjadi pada komunikasi WebSockets jika Anda nekat memanggil sintaks `delay(3000)` di dalam siklus `void loop()`?

- **Jawaban:** Fungsi `delay(3000)` bersifat **blocking** (menghentikan total seluruh eksekusi CPU selama 3 detik). Akibatnya:
 1. Mikrokontroler tidak akan bisa membaca status tombol fisik maupun memproses data sensor selama jeda tersebut.
 2. Koneksi WebSocket akan mengalami *timeout* atau terputus karena server gagal merespons *ping/pong* atau *heartbeat* jaringan secara tepat waktu.
 3. Sistem rentan mengalami *Watchdog Timer (WDT) Reset* yang membuat NodeMCU melakukan *reboot* otomatis akibat tugas utama jaringan terhenti terlalu lama.

Latihan 3 (Modifikasi Front-End)

Soal: Analisislah fungsi JavaScript pada blok perusakan UI. Nama ID elemen HTML apa saja yang menjadi target injeksi/pengubahan teks?

- **Jawaban:** Berdasarkan skrip JavaScript pada praktikum, elemen ID yang menjadi target manipulasi DOM (Document Object Model) adalah:
 1. tempValue: Target untuk menyuntikkan nilai suhu terbaru ke dalam elemen `<span>`.
 2. toggleBtn: Target untuk mengubah teks tombol (Turn ON / Turn OFF) serta kelas CSS tombol (btn-on / btn-off).
 3. ledStatus: Target untuk mengubah teks status indikator LED (ON / OFF).

Latihan 4 (Pengembangan Telemetry Sensor - Kelembapan)

Soal: Modifikasi program agar sistem dapat membaca dan menampilkan data kelembapan secara *real-time*.

- **Kode Modifikasi:**
 - **Sisi C++ (Back-End):**

C++

// Tambahkan variabel penampung kelembapan di atas

String currentHum = "--";

// Di dalam timer non-blocking millis() untuk DHT:

```
if((millis() - lastTime) > 3000) {
  float t = dht.readTemperature();
  float h = dht.readHumidity(); // Baca kelembapan
  if(!isnan(t) && !isnan(h)) {
    currentTemp = String(t);
    currentHum = String(h); // Simpan ke cache
    notifyClients();
  }
  lastTime = millis();
}
```

// Modifikasi fungsi notifyClients():

```
void notifyClients() {
  String jsonString = "{\"led\":\"" + String(ledState ? 1 : 0) + "\", ";
  jsonString += "\"suhu\":\"" + currentTemp + "\", ";
  jsonString += "\"hum\":\"" + currentHum + "\"}"; // Tambah properti hum
  ws.textAll(jsonString);
}
```

- **Sisi HTML (Front-End):**

HTML

<!-- Tambahkan card baru di dalam body -->

<div class="card">

<h2>Kelembapan: -- %</h2>

</div>

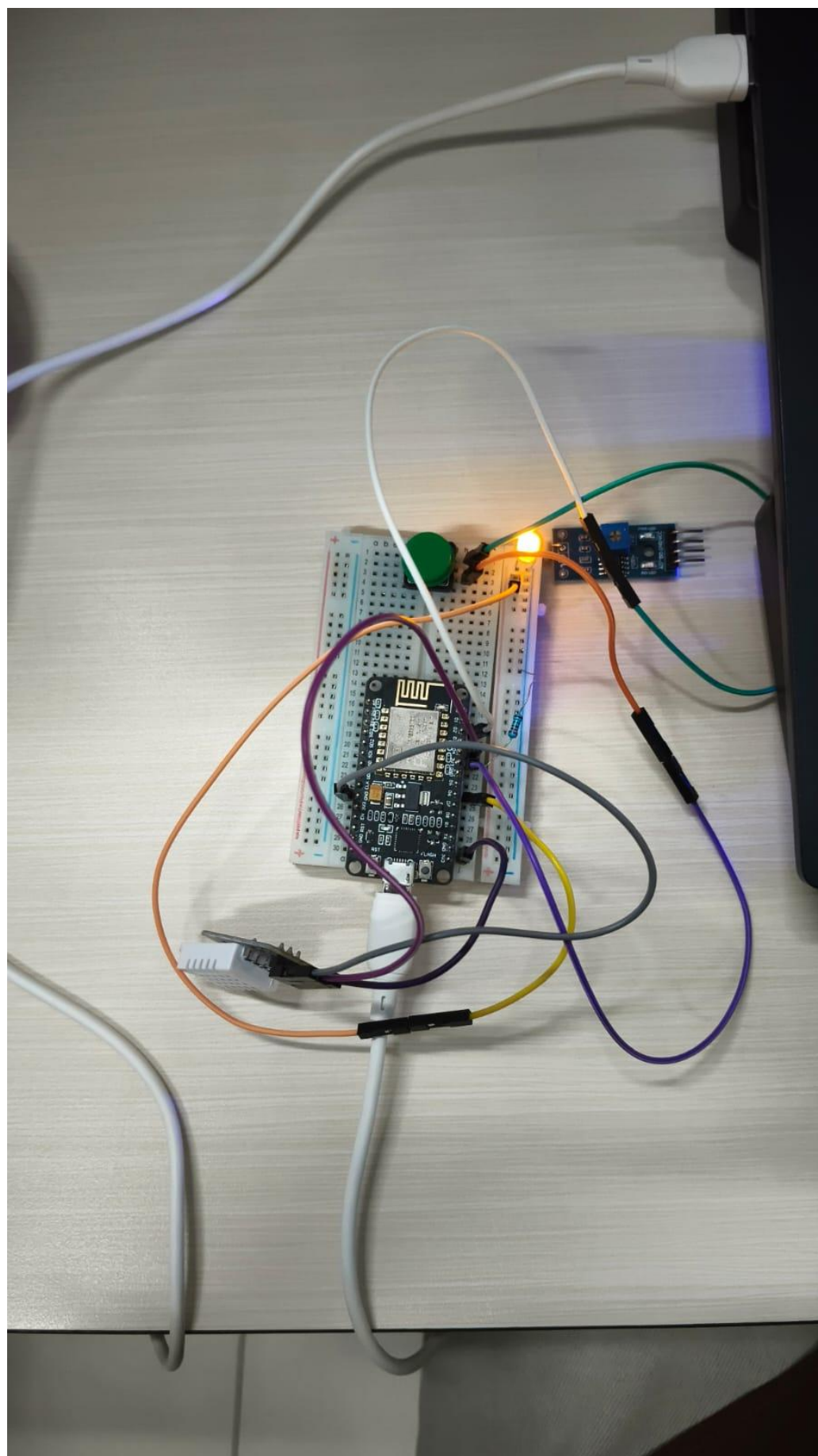
- **Sisi JavaScript (Front-End):**

JavaScript

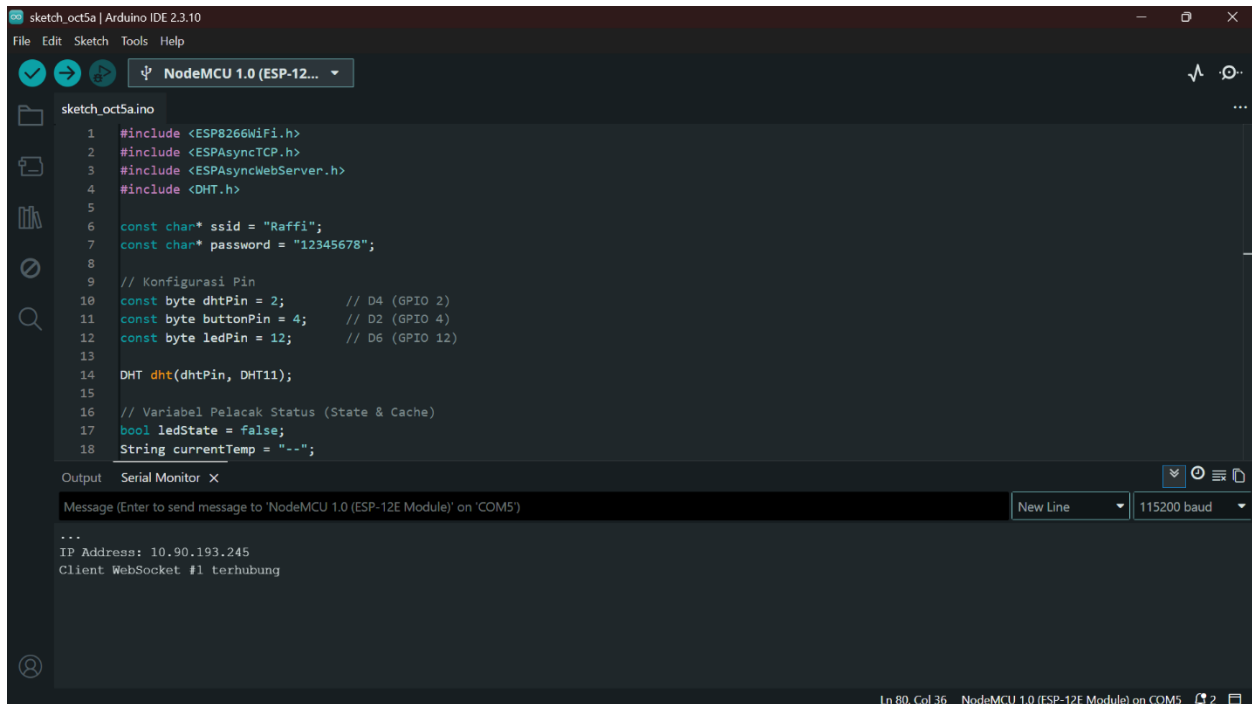
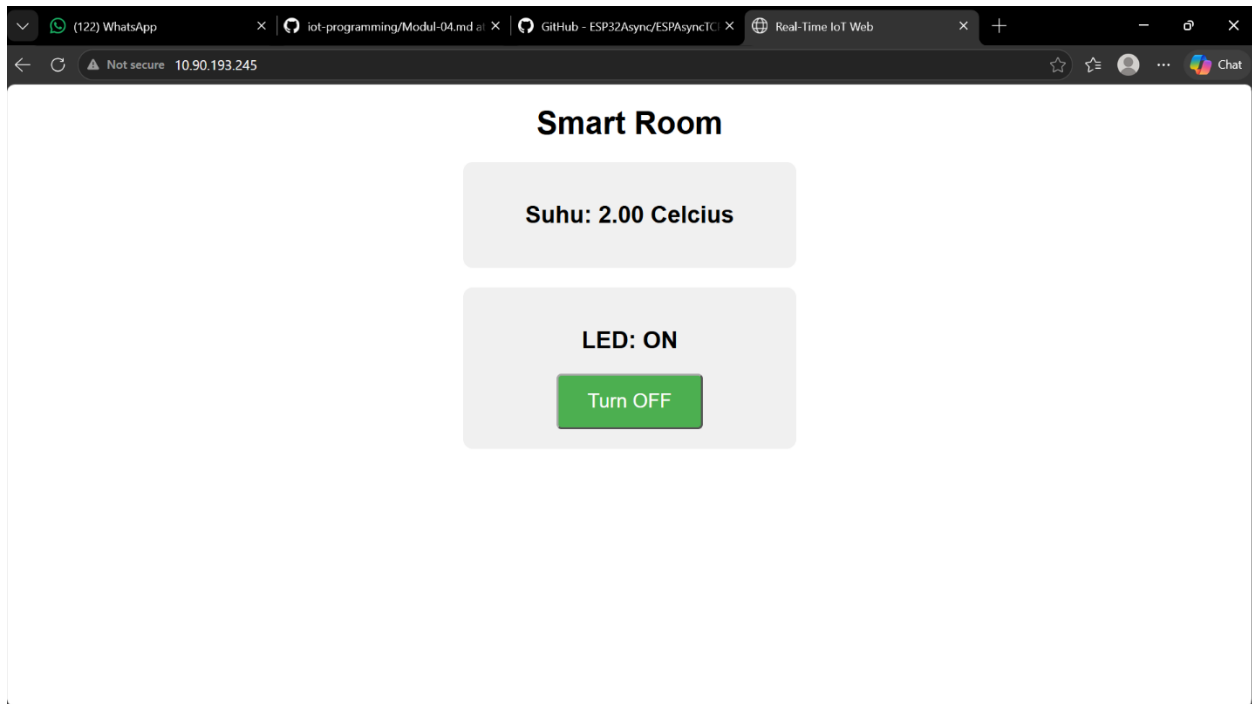
// Di dalam fungsi onMessage(event):

```
if(dataObj.hum !== undefined) {  
    document.getElementById('humValue').innerHTML = dataObj.hum;  
}
```

RANGKAIAN:



HASIL RUNNING WEB & KODE PEMOGRAMAN:



TUGAS MANDIRI :

3. Penyelesaian Tugas: Sistem Kendali Intensitas Cahaya (PWM Slider)

Berikut adalah tambahan kode untuk mengimplementasikan fitur PWM Slider (Dimmer LED) yang berjalan berdampingan secara mulus dengan sistem Praktikum 1.

1. Tambahan pada HTML (Front-End):

Sisipkan elemen card dan slider di bawah kendali LED pada kode index_html:

HTML

```
<div class="card">
  <h2>PWM Dimmer</h2>
  <input type="range" min="0" max="1023" value="0" id="pwmSlider"
oninput="sendPWM(this.value)">
  <p>Nilai: <span id="pwmValue">0</span></p>
</div>
```

Dan tambahkan fungsi JavaScript ini di dalam tag <script>:

JavaScript

```
function sendPWM(val) {
  document.getElementById('pwmValue').innerHTML = val;
  websocket.send("pwm," + val); // Kirim string berformat khusus ke server
}
```

2. Tambahan pada C++ (Back-End / NodeMCU):

- Definisikan pin LED PWM di bagian atas program:

C++

```
const byte pwmLedPin = 5; // D1 (GPIO 5) untuk PWM LED
int currentPwmValue = 0;
```

- Di dalam setup(), tambahkan inisialisasi pin:

C++

```
pinMode(pwmLedPin, OUTPUT);
```

- Modifikasi fungsi handleWebSocketMessage untuk memecah (*parsing*) pesan yang diawali dengan "pwm,":

C++

```
void handleWebSocketMessage(void *arg, uint8_t *data, size_t len) {
  AwsFrameInfo *info = (AwsFrameInfo*)arg;
  if (info->final && info->index == 0 && info->len == len && info->opcode == WS_TEXT) {
    data[len] = 0;
    String message = (char*)data; // Ubah ke objek String

    if (strcmp((char*)data, "toggle") == 0) {
      ledState = !ledState;
      notifyClients();
    }
    else if (message.startsWith("pwm,") {
      // Ambil angka setelah koma
      int pwmVal = message.substring(4).toInt();
      currentPwmValue = pwmVal;
    }
  }
}
```

```

    analogWrite(pwmLedPin, currentPwmValue); // Eksekusi PWM ke hardware
  }
}
}

```

KODE PROGRAM:

```

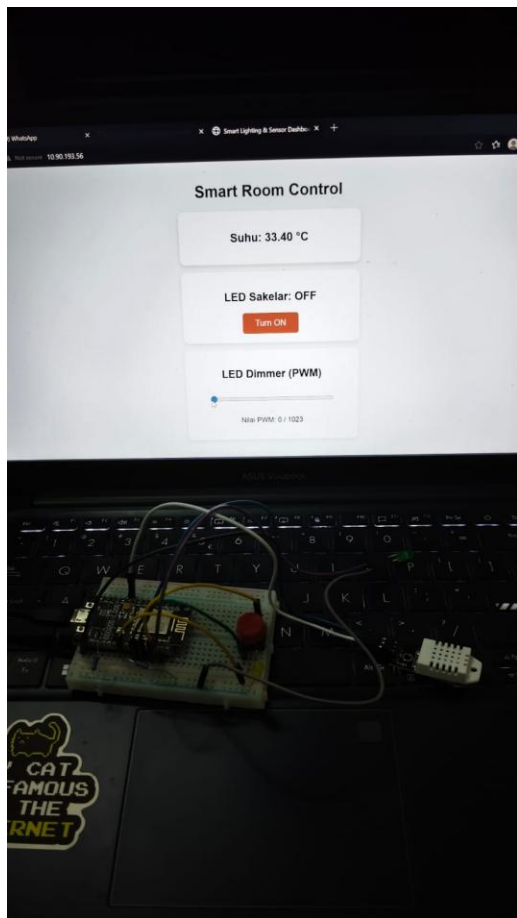
sketch_oct5a.ino
1  #include <ESP8266WiFi.h>
2  #include <ESPAsyncTCP.h>
3  #include <ESPAsyncWebServer.h>
4  #include <DHT.h>
5
6  const char* ssid = "Raffi";
7  const char* password = "12345678";
8
9  // Konfigurasi Pin
10 const byte dhtPin = 2;      // D4 (GPIO 2)
11 const byte buttonPin = 4;   // D2 (GPIO 4)
12 const byte ledPin = 12;     // D6 (GPIO 12) - On/Off LED
13 const byte pwmLedPin = 5;   // D1 (GPIO 5) - Dimmer PWM LED
14
15 DHT dht(dhtPin, DHT22);
16
17 // Variabel Status & Cache
18 bool ledState = false;
19 int pwmValue = 0;
20 String currentTemp = "--";
21 int buttonState;
22 int lastButtonState = LOW;
23 unsigned long lastDebounceTime = 0;
24 const unsigned long debounceDelay = 50;
25 unsigned long lastTime = 0;
26
27 // Inisialisasi Server & WebSocket
28 AsyncWebServer server(80);

```

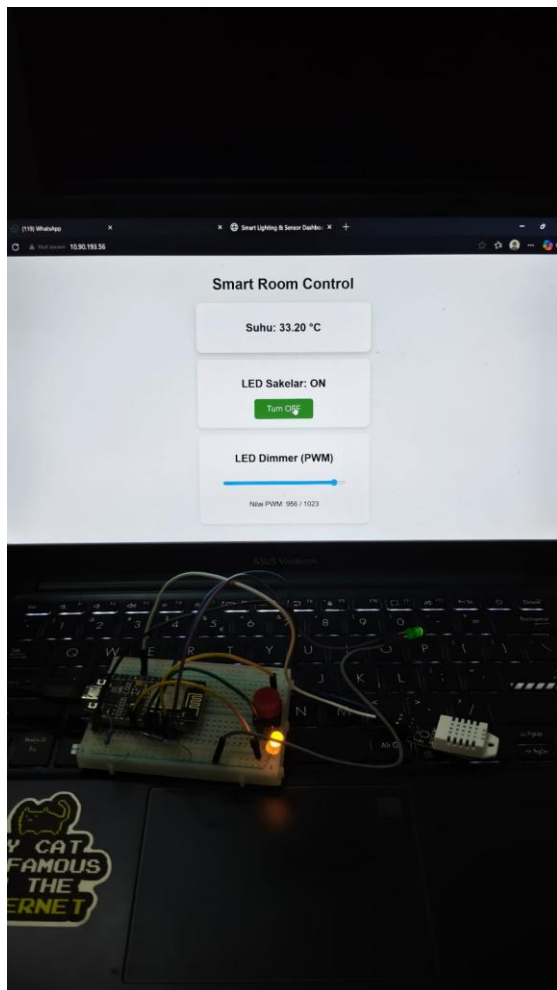
Output

NodeMCU 1.0 (ESP-12E Module) on COM5 [not connected]

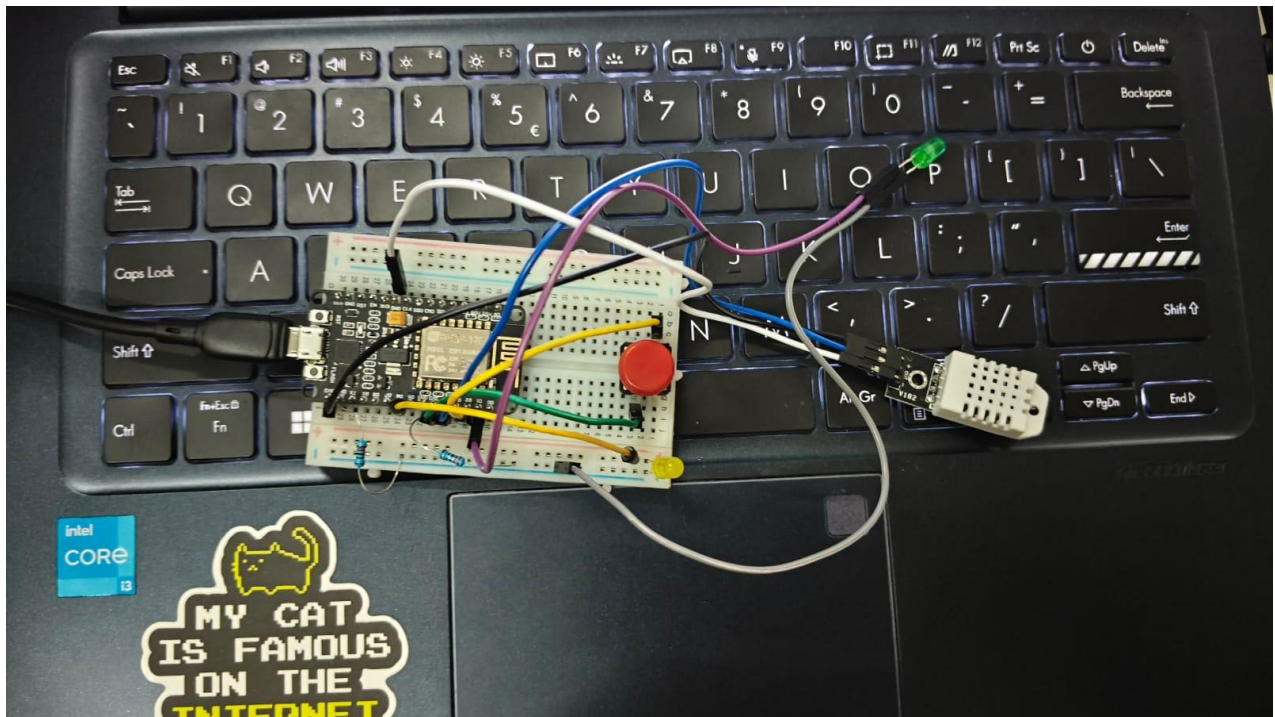
HASIL KETIKA LED OFF :



HASIL KETIKA LED ON :



RANGKAIAN :



MODUL 5 :

NAMA KELOMPOK :

1. Raffi Maulana Abdillah – 25782021
2. M. Khoiru Nizar – 25782018
3. M. Romzi – 25782015
4. M. Daffa Al-Barru - 25782017

PRAKTIKKUM:

5.5.1 :

5.5.1 Praktikum 1: Identifikasi MAC Address Perangkat NodeMCU

Sebelum membangun jaringan ESP-NOW, setiap anggota kelompok wajib membaca dan mencatat alamat fisik (*MAC Address*) dari masing-masing *board* NodeMCU.

Langkah Kerja Pemrograman:

1. Buat berkas baru (*New Sketch*) pada Arduino IDE.

2. Tuliskan kode program pembaca MAC Address berikut:

```
#include <ESP8266WiFi.h>

void setup() {
  Serial.begin(115200);
  Serial.println();

  // Mengaktifkan mode Station agar MAC address antarmuka STA aktif
  WiFi.mode(WIFI_STA);
  WiFi.disconnect();

  Serial.print("ESP8266 MAC Address: ");
  Serial.println(WiFi.macAddress());
}

void loop() {
  // Tidak ada proses berulang
}
```

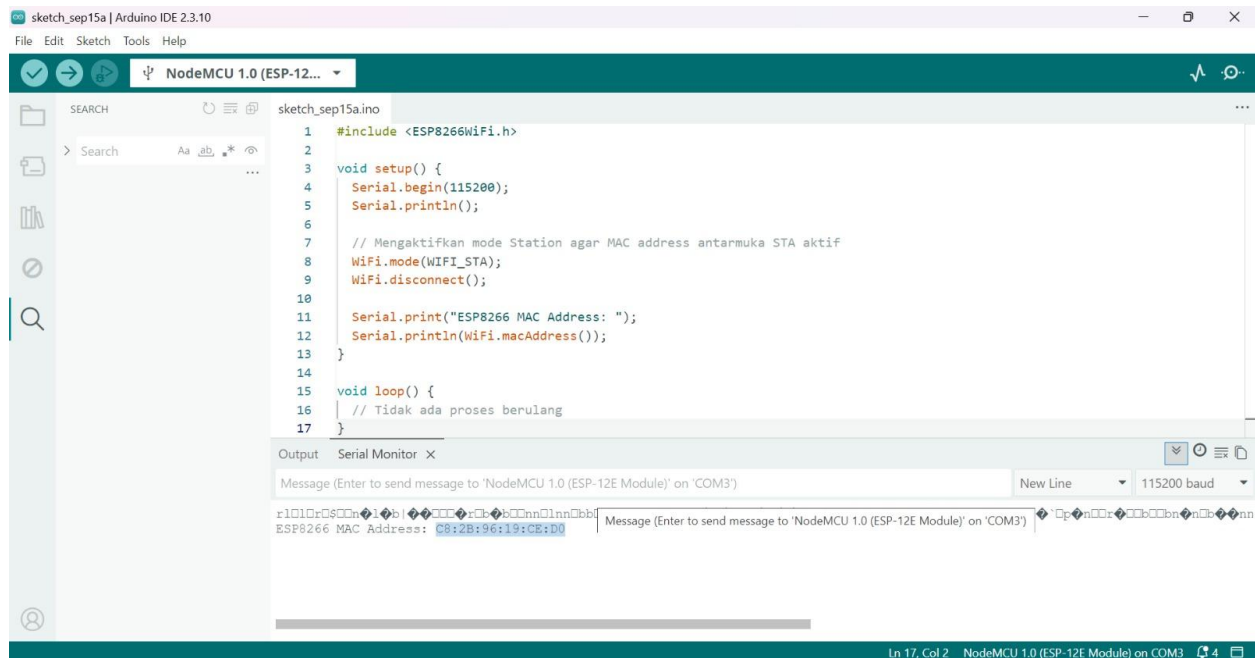
3. **Upload** program ke masing-masing *board* NodeMCU dalam kelompok Anda secara bergantian.
4. Buka **Serial Monitor** pada kecepatan **115200 baud**, lalu tekan tombol fisik **RST** pada *board*.
5. Catat 6 pasang digit heksadesimal yang muncul di layar (contoh: BC:FF:4D:23:45:90).
6. Beri label fisik pada *board* (misal: "Node 1", "Node 2", "Hub Pusat") beserta alamat MAC masing-masing agar tidak tertukar saat perakitan.

Penjelasan Singkat Kode:

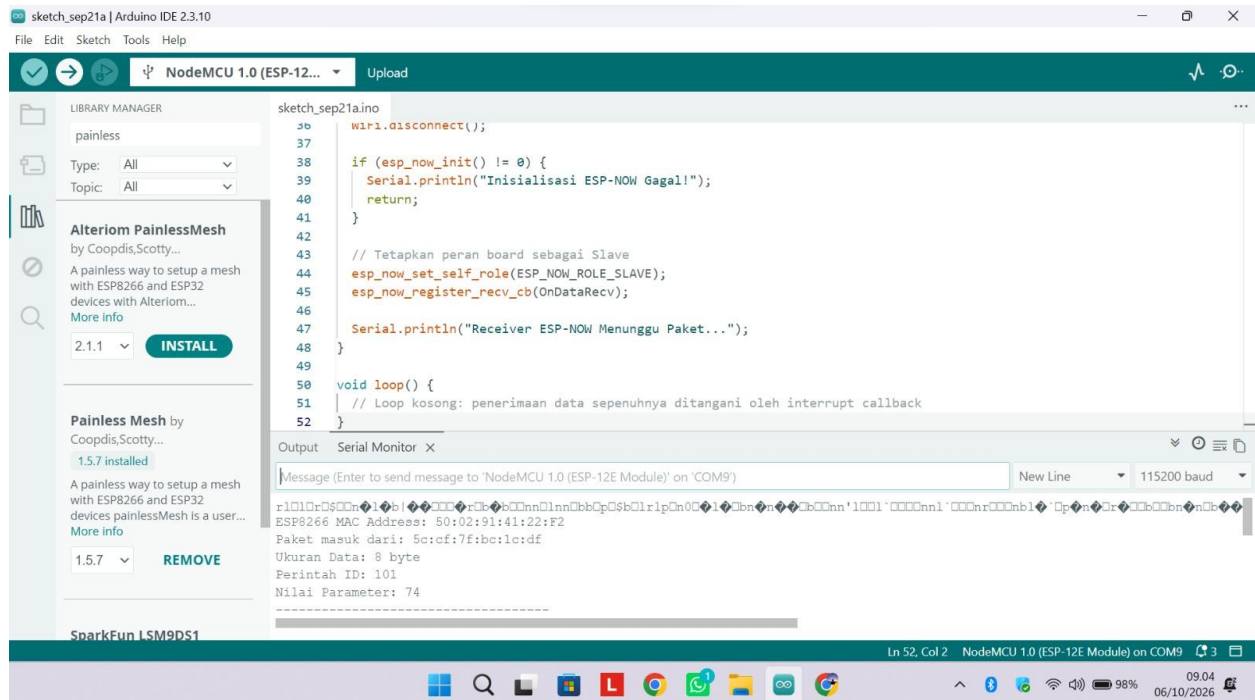
- `WiFi.mode(WIFI_STA);` : Menyiapkan modul radio Wi-Fi internal ke dalam mode *Station* agar alamat perangkat keras (*Hardware MAC*) siap dialokasikan oleh SDK.
- `WiFi.disconnect();` : Memastikan mikrokontroler tidak mencoba melakukan proses penyambungan ke jaringan luar.
- `WiFi.macAddress();` : Mengambil string alamat fisik 48-bit bawaan pabrik yang bersifat unik secara global.

KODE DAN HASIL OUTPUT:

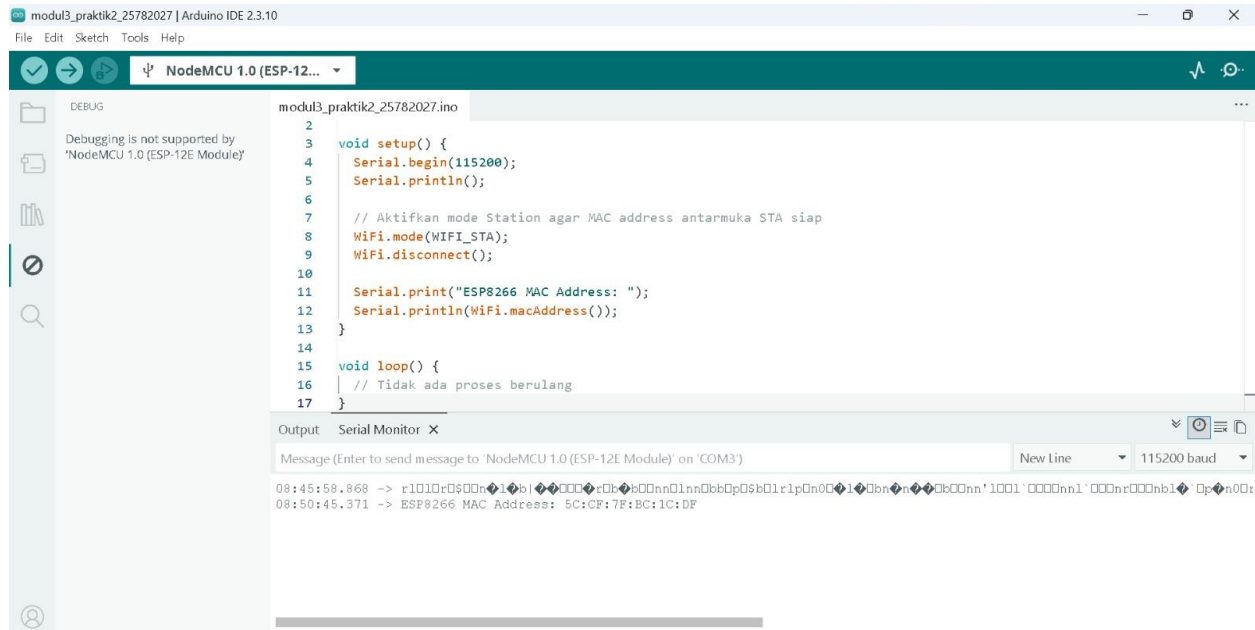
MAC 1 :



MAC 2 :



MAC 3 :



PRAKTIKKUM 5.5.2 :

5.5.2 Praktikum 2: Komunikasi Nirkabel One-to-Many (ESP-NOW)

Skenario: Satu unit NodeMCU bertindak sebagai **Controller / Sender**, menyiarkan paket data instruksi yang sama secara langsung ke dua unit NodeMCU lain yang bertindak sebagai **Slave / Receiver** tanpa perantara router.

Langkah Kerja Pemrograman Sisi Pengirim (Controller):

1. Masukkan alamat MAC milik Board Penerima 1 dan Board Penerima 2 ke dalam variabel receiver1 dan receiver2.
2. Format tanda titik dua (:) diubah menjadi bilangan heksadesimal berawalan 0x yang dipisahkan oleh tanda koma (contoh: BC:FF:4D:23:45:90 → {0xBC, 0xFF, 0x4D, 0x23, 0x45, 0x90}).
3. Tuliskan kode berikut dan unggah ke **NodeMCU 1**:

```
#include <ESP8266WiFi.h>
#include <espnow.h>
```

// GANTI dengan MAC Address Board Penerima 1 dan Board Penerima 2 Anda!

```
uint8_t receiver1[] = {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF};
uint8_t receiver2[] = {0xAA, 0xBB, 0xCC, 0xDD, 0xEE, 0xFF};
```

// Definisi struktur data paket instruksi

```
typedef struct struct_pesan {
    int perintahId;
    int nilaiParameter;
} struct_pesan;
```

```
struct_pesan paketKirim;
```

```
unsigned long previousMillis = 0;
```

```

const long interval = 2000; // Transmisi berkala setiap 2 detik

// Callback otomatis saat paket selesai dipancarkan radio
void OnDataSent(uint8_t *mac_addr, uint8_t sendStatus) {
    char macStr[18];
    snprintf(macStr, sizeof(macStr), "%02x:%02x:%02x:%02x:%02x:%02x",
             mac_addr[0], mac_addr[1], mac_addr[2], mac_addr[3], mac_addr[4], mac_addr[5]);

    Serial.print("Kirim paket ke: ");
    Serial.print(macStr);
    Serial.print(" | Status: ");
    if (sendStatus == 0) {
        Serial.println("Berhasil Diterima");
    } else {
        Serial.println("Gagal (Tidak Terjangkau)");
    }
}

void setup() {
    Serial.begin(115200);
    WiFi.mode(WIFI_STA);
    WiFi.disconnect();

    // Inisialisasi stack protokol ESP-NOW
    if (esp_now_init() != 0) {
        Serial.println("Gagal menginisialisasi ESP-NOW!");
        return;
    }

    // Tetapkan peran board sebagai Controller
    esp_now_set_self_role(ESP_NOW_ROLE_CONTROLLER);
    esp_now_register_send_cb(OnDataSent);

    // Daftarkan kedua board penerima ke tabel peer internal
    esp_now_add_peer(receiver1, ESP_NOW_ROLE_SLAVE, 1, NULL, 0);
    esp_now_add_peer(receiver2, ESP_NOW_ROLE_SLAVE, 1, NULL, 0);

    Serial.println("Controller ESP-NOW Siap!");
}

void loop() {
    unsigned long currentMillis = millis();
    if (currentMillis - previousMillis >= interval) {
        previousMillis = currentMillis;

        paketKirim.perintahId = 101;
        paketKirim.nilaiParameter = random(10, 100);

        // Argumen 0 berarti mengirimkan paket ke seluruh peer yang terdaftar
    }
}

```

```

    esp_now_send(0, (uint8_t *) &paketKirim, sizeof(paketKirim));
}
}

```

Langkah Kerja Pemrograman Sisi Penerima (Slave):

1. Unggah kode sketsa berikut ke **NodeMCU 2** dan **NodeMCU 3**.
2. Board penerima tidak perlu mengetahui alamat MAC milik Controller.

```
#include <ESP8266WiFi.h>
```

```
#include <espnow.h>
```

```
// Struktur data penerima (wajib identik dengan sisi pengirim)
```

```
typedef struct struct_pesana {
    int perintahId;
    int nilaiParameter;
} struct_pesana;
```

```
struct_pesana paketTerima;
```

```
// Callback otomatis saat antenna mendeteksi paket masuk
```

```
void OnDataRecv(uint8_t *mac_addr, uint8_t *incomingData, uint8_t len) {
    char macStr[18];
    snprintf(macStr, sizeof(macStr), "%02x:%02x:%02x:%02x:%02x:%02x",
             mac_addr[0], mac_addr[1], mac_addr[2], mac_addr[3], mac_addr[4], mac_addr[5]);
```

```
// Salin buffer memori radio langsung ke struktur data variabel
memcpy(&paketTerima, incomingData, sizeof(paketTerima));
```

```
Serial.print("Paket masuk dari: ");
Serial.println(macStr);
Serial.print("Ukuran Data: ");
Serial.print(len);
Serial.println(" byte");
Serial.print("Perintah ID: ");
Serial.println(paketTerima.perintahId);
Serial.print("Nilai Parameter: ");
Serial.println(paketTerima.nilaiParameter);
Serial.println("-----");
}
```

```
void setup() {
    Serial.begin(115200);
    WiFi.mode(WIFI_STA);
    WiFi.disconnect();
```

```
if (esp_now_init() != 0) {
    Serial.println("Inisialisasi ESP-NOW Gagal!");
    return;
}
```

```

// Tetapkan peran board sebagai Slave
esp_now_set_self_role(ESP_NOW_ROLE_SLAVE);
esp_now_register_recv_cb(OnDataRecv);

Serial.println("Receiver ESP-NOW Menunggu Paket...");
}

void loop() {
  // Loop kosong: penerimaan data sepenuhnya ditangani oleh interrupt callback
}

```

Penjelasan Fungsi Khusus ESP-NOW:

- `esp_now_init()` : Menginisialisasi *firmware stack* ESP-NOW pada chip ESP8266. Mengembalikan nilai 0 jika berhasil.
- `esp_now_set_self_role()` : Mengonfigurasi peran operasional NodeMCU (`ESP_NOW_ROLE_CONTROLLER` untuk pengirim, `ESP_NOW_ROLE_SLAVE` untuk penerima).
- `esp_now_register_send_cb(OnDataSent)` : Mendaftarkan fungsi *callback* yang akan dipanggil sesaat setelah gelombang radio memancarkan paket untuk memastikan laporan ACK diterima atau gagal.
- `esp_now_register_recv_cb(OnDataRecv)` : Mendaftarkan fungsi *callback* interupsi saat paket radio masuk ke antena penerima.
- `esp_now_add_peer(mac, role, channel, key, key_len)` : Mendaftarkan perangkat tujuan ke dalam tabel routing memori chip.
- `esp_now_send(0, data, size)` : Mengirimkan muatan data. Memasukkan angka 0 pada argumen pertama berarti paket dikirimkan berurutan ke **seluruh** *peer* yang terdaftar.
- `memcpy(&paketTerima, incomingData, sizeof(paketTerima))` : Menyalin deretan byte mentah dari penyangga memori (*buffer*) ke dalam variabel struktur `paketTerima`.

NODE MCU UTAMA :

The screenshot shows the Arduino IDE interface. The top bar indicates the board is 'NodeMCU 1.0 (ESP-12E Module)'. The code editor displays the following code:

```

1 #include <ESP8266WiFi.h>
2 #include <espnow.h>
3
4 // GANTI dengan MAC Address Board Penerima 1 dan Board Penerima 2 Andai
5 uint8_t receiver1[] = {0xC8, 0x2B, 0x96, 0x19, 0xCE, 0xD0};
6 uint8_t receiver2[] = {0x50, 0x02, 0x91, 0x41, 0x22, 0xF2};
7
8 // Definisi struktur data paket instruksi
9 typedef struct struct_pesan {
10   int perintahId;
11   int nilaiParameter;
12 } struct_pesan;
13
14 struct_pesan paketKirim;
15
16 unsigned long previousMillis = 0;

```

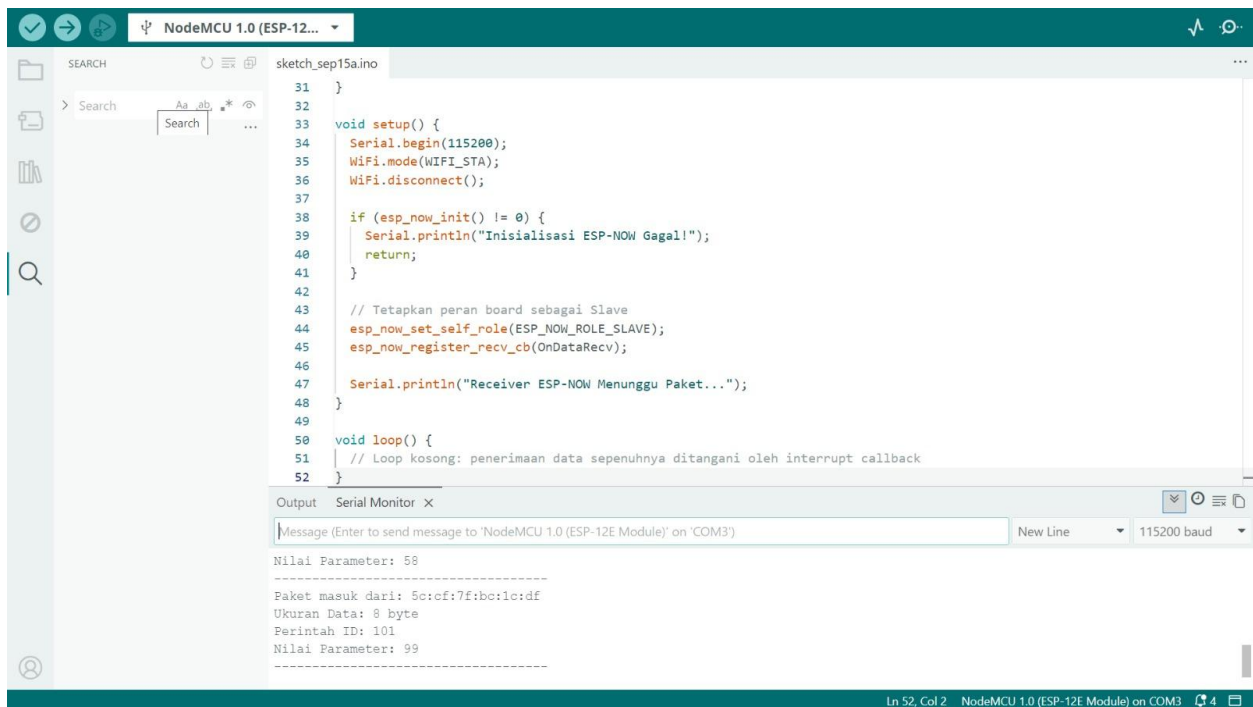
The serial monitor at the bottom shows the following output:

```

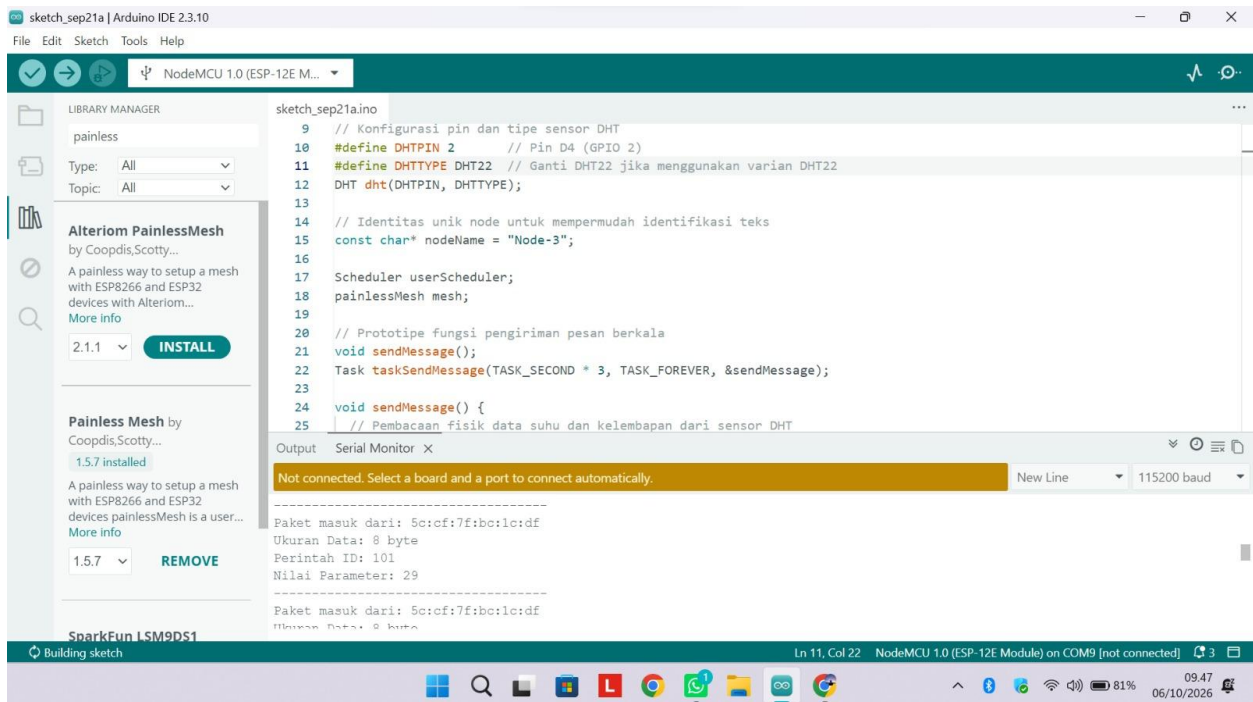
09:04:27.697 -> Kirim paket ke: c8:2b:96:19:ce:d0 | Status: Berhasil Diterima
09:04:29.721 -> Kirim paket ke: 50:02:91:41:22:f2 | Status: Berhasil Diterima
09:04:29.731 -> Kirim paket ke: c8:2b:96:19:ce:d0 | Status: Berhasil Diterima
09:04:29.731 -> Kirim paket ke: 50:02:91:41:22:f2 | Status: Berhasil Diterima
09:04:31.688 -> Kirim paket ke: c8:2b:96:19:ce:d0 | Status: Berhasil Diterima
09:04:31.722 -> Kirim paket ke: 50:02:91:41:22:f2 | Status: Berhasil Diterima
09:04:33.687 -> Kirim paket ke: c8:2b:96:19:ce:d0 | Status: Berhasil Diterima
09:04:33.728 -> Kirim paket ke: 50:02:91:41:22:f2 | Status: Berhasil Diterima

```

NODE MCU 2 :



NODE MCU 3 :



5.5.3 Praktikum 3: Pengumpulan Data Telemetri Many-to-One (ESP-NOW)

Skenario: Dua node sensor di lapangan memancarkan data pengukuran masing-masing ke sebuah node konsentrator pusat (*Aggregator Hub / Base Station*).

Langkah Kerja Pemrograman Sisi Node Pengirim:

1. Masukkan alamat MAC milik **Board Penerima Pusat (Hub)** pada variabel `hubMacAddress`.
2. Pada Board Pengirim 1, atur: `#define BOARD_ID 1`.
3. Pada Board Pengirim 2, atur: `#define BOARD_ID 2`.
4. Unggah kode program berikut:

```
#include <ESP8266WiFi.h>
#include <espnow.h>

// GANTI dengan MAC Address milik BOARD PENERIMA PUSAT (HUB)!
uint8_t hubMacAddress[] = {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF};

// Identitas unik node (Ubah angka ini untuk node yang berbeda!)
#define BOARD_ID 1

typedef struct struct_sensor {
    int id;
    float pembacaan1;
    float pembacaan2;
} struct_sensor;

struct_sensor dataKirim;

unsigned long prevTime = 0;
const unsigned long sendInterval = 3000; // Kirim data setiap 3 detik

void OnDataSent(uint8_t *mac_addr, uint8_t sendStatus) {
    Serial.print("Pengiriman Node #");
    Serial.print(BOARD_ID);
    Serial.println(sendStatus == 0 ? " -> Sukses Diterima Hub" : " -> Gagal Sampai");
}

void setup() {
    Serial.begin(115200);
    WiFi.mode(WIFI_STA);
    WiFi.disconnect();

    if (esp_now_init() != 0) {
        Serial.println("Inisialisasi ESP-NOW gagal");
        return;
    }

    esp_now_set_self_role(ESP_NOW_ROLE_CONTROLLER);
    esp_now_register_send_cb(OnDataSent);
```

```

// Daftarkan hub penerima tunggal
esp_now_add_peer(hubMacAddress, ESP_NOW_ROLE_SLAVE, 1, NULL, 0);
}

void loop() {
  unsigned long now = millis();
  if (now - prevTime >= sendInterval) {
    prevTime = now;

    dataKirim.id = BOARD_ID;
    dataKirim.pembacaan1 = random(250, 350) / 10.0; // Simulasi pembacaan suhu: 25.0 - 35.0 C
    dataKirim.pembacaan2 = random(400, 800) / 10.0; // Simulasi pembacaan kelembapan: 40.0 -
80.0 %

    esp_now_send(hubMacAddress, (uint8_t *) &dataKirim, sizeof(dataKirim));
  }
}

```

Langkah Kerja Pemrograman Sisi Hub Konsentrator:

Unggah kode sketsa berikut ke Board Penerima Pusat:

```
#include <ESP8266WiFi.h>
```

```
#include <espnow.h>
```

```

typedef struct struct_sensor {
  int id;
  float pembacaan1;
  float pembacaan2;
} struct_sensor;

```

```
struct_sensor dataTerima;
```

```

// Array penampung data terakhir: Index 0 untuk Node 1, Index 1 untuk Node 2
struct_sensor records[2];

```

```

void OnDataRecv(uint8_t *mac_addr, uint8_t *incomingData, uint8_t len) {
  memcpy(&dataTerima, incomingData, sizeof(dataTerima));

```

```
// Validasi ID agar tidak terjadi error index array out-of-bounds
```

```

if (dataTerima.id >= 1 && dataTerima.id <= 2) {
  int index = dataTerima.id - 1;
  records[index].id = dataTerima.id;
  records[index].pembacaan1 = dataTerima.pembacaan1;
  records[index].pembacaan2 = dataTerima.pembacaan2;

```

```
Serial.println("=====");
```

```
Serial.printf("DATA TERBARU DARI NODE #%d\n", dataTerima.id);
```

```
Serial.printf("Parameter 1 : %.2f\n", records[index].pembacaan1);
```

```

        Serial.printf("Parameter 2 : %.2f\n", records[index].pembacaan2);
        Serial.println("=====");
    }
}

void setup() {
    Serial.begin(115200);
    WiFi.mode(WIFI_STA);
    WiFi.disconnect();

    if (esp_now_init() != 0) {
        Serial.println("Gagal Inisialisasi ESP-NOW Hub!");
        return;
    }

    esp_now_set_self_role(ESP_NOW_ROLE_SLAVE);
    esp_now_register_recv_cb(OnDataRecv);

    Serial.println("Hub Konsentrator Siap Menerima Data Sensor...");
}

void loop() {
    // Loop bebas menangani tugas lain
}

```

NODE PENGIRIM 1 :

The screenshot shows the Arduino IDE interface with the following code in the sketch editor:

```

1 #include <ESP8266WiFi.h>
2 #include <espnow.h>
3
4 // GANTI dengan MAC Address milik BOARD PENERIMA PUSAT (HUB)!
5 uint8_t hubMacAddress[] = {0x5C, 0xCF, 0x7F, 0xBC, 0x1C, 0xDF};
6
7 // Identitas unik node (Ubah angka ini untuk node yang berbeda!)
8 #define BOARD_ID 1
9
10 typedef struct struct_sensor {
11     int id;
12     float pembacaan1;
13     float pembacaan2;
14 } struct_sensor;
15
16 struct_sensor dataKirim;
17
18 unsigned long prevTime = 0;
19 const unsigned long sendInterval = 3000; // Kirim data setiap 3 detik
20
21 void OnDataSent(uint8_t *mac_addr, uint8_t sendStatus) {
22     Serial.print("Pengiriman Node #");

```

The Serial Monitor shows the following output:

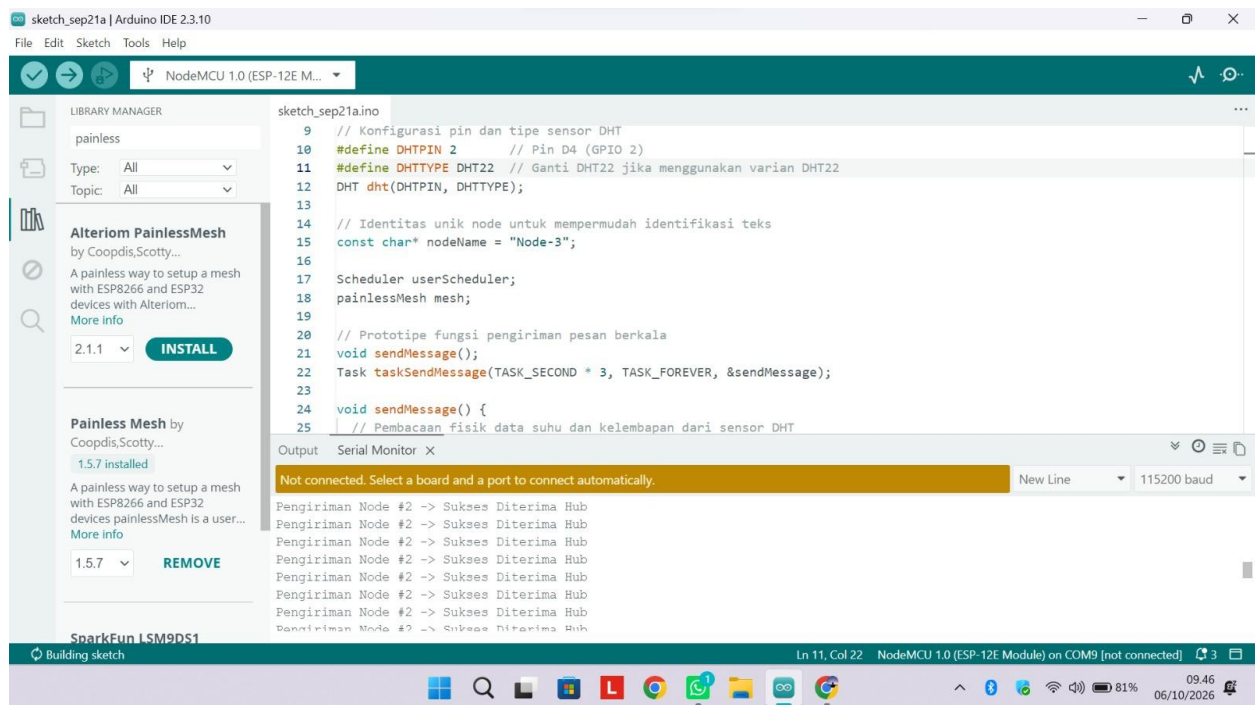
```

Pengiriman Node #1 -> Sukses Diterima Hub
Pengiriman Node #1 -> Sukses Diterima Hub
Pengiriman Node #1 -> Sukses Diterima Hub
Pengiriman Node #1 -> Sukses Diterima Hub
Pengiriman Node #1 -> Sukses Diterima Hub
Pengiriman Node #1 -> Sukses Diterima Hub
Pengiriman Node #1 -> Sukses Diterima Hub

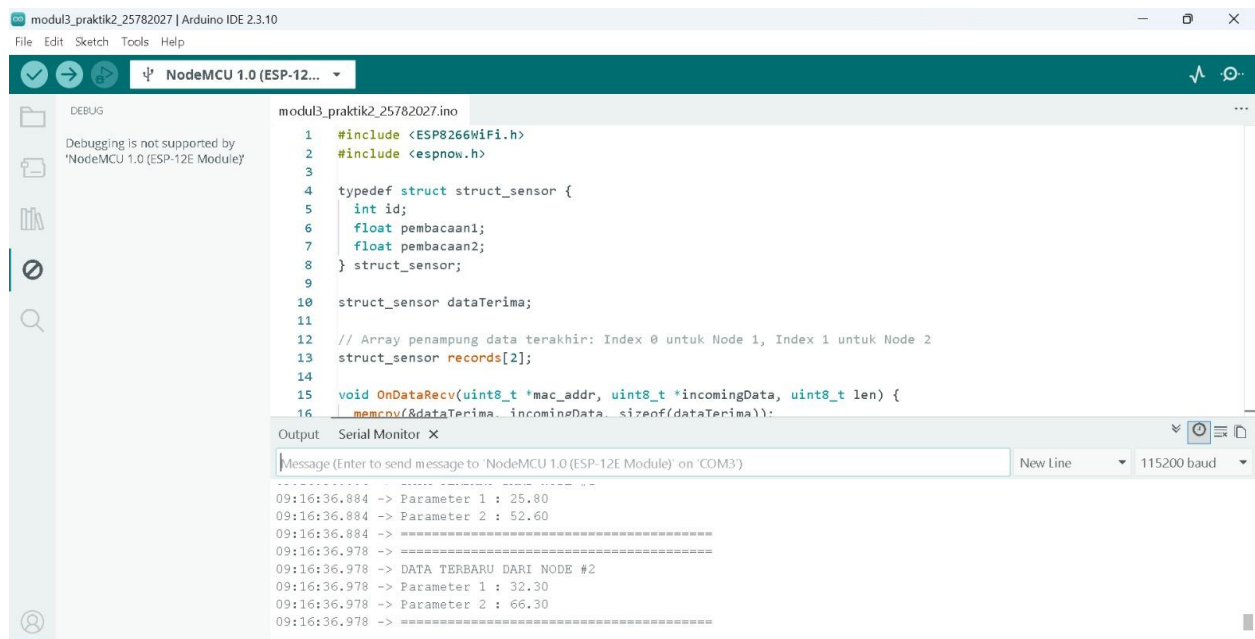
```

The status bar at the bottom indicates: Building sketch, Ln 8, Col 19, NodeMCU 1.0 (ESP-12E Module) on COM3, 4.

NODE PENGIRIM 2 :



NODE PENERIMA PUSAT UTAMA :



5.5.4 Praktikum 4: Jaringan Sensor Ad-Hoc Multi-Hop (painlessMesh)

Skenario: Seluruh board dalam kelompok dihubungkan ke sensor suhu & kelembapan riil (DHT11/DHT22) dan diintegrasikan ke dalam sebuah jaringan rantai otonom (*Mesh*). Setiap board secara berkala membaca sensor fisiknya, mengemas data telemetri ke dalam format **JSON**, dan menyiarkannya ke seluruh anggota jaringan tanpa perlu mengetahui alamat MAC masing-masing.

Instruksi Perangkaian Perangkat Keras:

- Pasang sensor **DHT11** / **DHT22** pada masing-masing board NodeMCU:
 - Pin **VCC** sensor ke pin **3V3 (3.3V)** NodeMCU.
 - Pin **GND** sensor ke pin **GND** NodeMCU.
 - Pin **DATA / OUT** sensor ke pin **D4 (GPIO 2)** NodeMCU.

Langkah Kerja Pemrograman:

1. Unggah sketsa berikut ke **seluruh board** dalam kelompok Anda.
2. Berikan nama pengenalan unik pada variabel nodeName di masing-masing board (misal: "Node-1", "Node-2", "Node-3").
3. Jika menggunakan varian DHT22, ubah #define DHTTYPE DHT11 menjadi DHT22.

```
#include <painlessMesh.h>
#include <DHT.h>
#include <ArduinoJson.h>

#define MESH_PREFIX "LabIoTMesh" // Nama jaringan Mesh bersama
#define MESH_PASSWORD "iotmeshpassword" // Kata sandi jaringan Mesh
#define MESH_PORT 5555 // Port komunikasi TCP Mesh

// Konfigurasi pin dan tipe sensor DHT
#define DHTPIN 2 // Pin D4 (GPIO 2)
#define DHTTYPE DHT11 // Ganti DHT22 jika menggunakan varian DHT22
DHT dht(DHTPIN, DHTTYPE);

// Identitas unik node untuk mempermudah identifikasi teks
const char* nodeName = "Node-1";

Scheduler userScheduler;
painlessMesh mesh;

// Prototipe fungsi pengiriman pesan berkala
void sendMessage();
Task taskSendMessage(TASK_SECOND * 3, TASK_FOREVER, &sendMessage);

void sendMessage() {
    // Pembacaan fisik data suhu dan kelembapan dari sensor DHT
    float suhu = dht.readTemperature();
    float kelembapan = dht.readHumidity();

    // Validasi pembacaan sensor
    if (isnan(suhu) || isnan(kelembapan)) {
        Serial.println("[DHT Error] Gagal membaca data dari sensor DHT!");
        return;
    }
}

// Rangkai data telemetri ke dalam format JSON menggunakan ArduinoJson
StaticJsonDocument<200> doc;
doc["node"] = nodeName;
```

```

doc["chipId"] = mesh.getNodeId();
doc["suhu"] = suhu;
doc["kelembapan"] = kelembapan;

String msg;
serializeJson(doc, msg);

// Siarkan paket JSON ke seluruh jaringan mesh (multi-hop broadcast)
mesh.sendBroadcast(msg);

Serial.print("[KIRIM MESH] ");
Serial.println(msg);
}

// Callback otomatis saat menerima pesan dari node manapun di jaringan mesh
void receivedCallback(uint32_t from, String &msg) {
    // Parsing muatan teks JSON yang diterima
    StaticJsonDocument<200> doc;
    DeserializationError error = deserializeJson(doc, msg);

    if (!error) {
        const char* sender = doc["node"];
        uint32_t chipId = doc["chipId"];
        float suhu = doc["suhu"];
        float kelembapan = doc["kelembapan"];

        Serial.println("=====");
        Serial.printf("[TERIMA DARI] %s (Node ID: %u | Chip ID: %u)\n", sender, from, chipId);
        Serial.printf("Suhu : %.2f °C\n", suhu);
        Serial.printf("Kelembapan : %.2f %%\n", kelembapan);
        Serial.println("=====");
    } else {
        Serial.printf("[TERIMA DATA MENTAH DARI %u]: %s\n", from, msg.c_str());
    }
}

// Callback otomatis saat ada perangkat baru bergabung ke mesh
void newConnectionCallback(uint32_t nodeId) {
    Serial.printf("--> Koneksi Baru Terdeteksi! Node ID: %u\n", nodeId);
}

// Callback otomatis saat topologi rantai mesh berubah (node keluar/masuk)
void changedConnectionCallback() {
    Serial.println("--> Topologi rantai mesh telah diperbarui");
}

void setup() {
    Serial.begin(115200);

```

```

// Inisialisasi sensor DHT
dht.begin();

// Atur level log debugging (hanya tampilkan error dan status startup)
mesh.setDebugMsgTypes(ERROR | STARTUP);

// Inisialisasi jaringan mesh
mesh.init(MESH_PREFIX, MESH_PASSWORD, &userScheduler, MESH_PORT);

// Daftarkan event callback
mesh.onReceive(&receivedCallback);
mesh.onNewConnection(&newConnectionCallback);
mesh.onChangedConnections(&changedConnectionCallback);

// Jadwalkan tugas pengiriman data berkala menggunakan TaskScheduler
userScheduler.addTask(taskSendMessage);
taskSendMessage.enable();

Serial.printf("Mesh Node [%s] Berjalan. Menunggu pembentukan topologi...\n", nodeName);
}

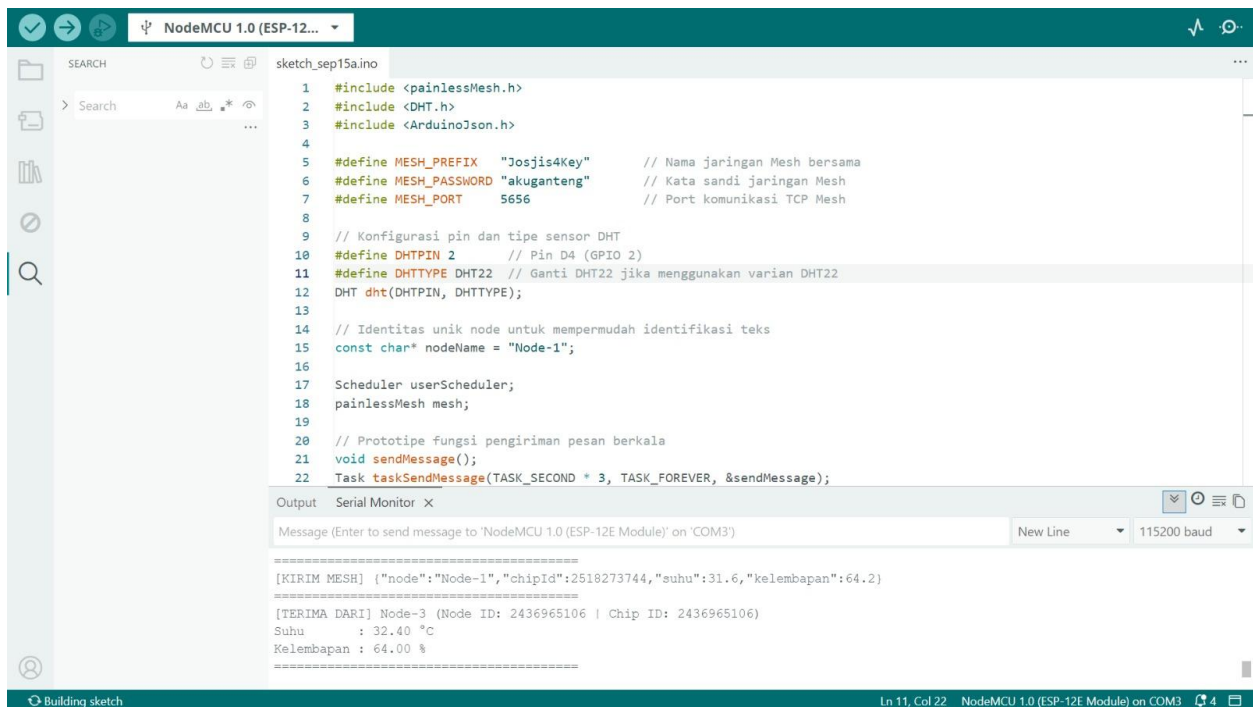
void loop() {
  // Wajib dipanggil rutin untuk memproses perutean paket dan sinkronisasi waktu
  mesh.update();
}

```

Penjelasan Kode Khusus **painlessMesh** & Integrasi JSON:

- `dht.readTemperature()` & `dht.readHumidity()` : Membaca nilai fisik suhu (° C) dan kelembapan () dari sensor DHT11/DHT22.
- `serializeJson(doc, msg)` : Mengubah struktur objek JSON menjadi deretan teks string ringkas siap siar.
- `deserializeJson(doc, msg)` : Membedah string teks JSON yang masuk kembali menjadi variabel numerik terpisah.
- `painlessMesh mesh;` : Menginisialisasi objek kontrol jaringan mesh yang menangani pembentukan soft-AP dan koneksi STA secara simultan di latar belakang.
- `mesh.getNodeId()` : Menghasilkan ID unik numerik berbasis MAC address bawaan chip ESP8266.
- `mesh.sendBroadcast(msg)` : Menyiarkan paket data teks ke seluruh node di dalam jaringan mesh. Paket akan diteruskan (*relayed*) otomatis oleh node perantara jika node target berada di luar jangkauan radio langsung.
- `TaskScheduler (userScheduler)` : Pengganti fungsi `delay()`. Di dalam jaringan mesh, penggunaan `delay()` dilarang keras karena akan membekukan tugas latar belakang perutean paket (*routing packet task*), yang dapat menyebabkan runtuhnya topologi mesh.
- `mesh.update()` : Menjalankan pemeliharaan koneksi, sinkronisasi jam internal mesh, dan pertukaran tabel rute secara kontinu di dalam siklus `loop()`.

NODE 1 :



The screenshot shows the Arduino IDE interface with the file 'sketch_sep15a.ino' open. The code defines a Node-1 with a unique ID, a password, and a port. It includes a DHT sensor and a scheduler. The Serial Monitor shows the output of the code, including the node's identity and sensor data.

```
1 #include <painlessMesh.h>
2 #include <DHT.h>
3 #include <ArduinoJson.h>
4
5 #define MESH_PREFIX "Josjis4Key" // Nama jaringan Mesh bersama
6 #define MESH_PASSWORD "akuganteng" // Kata sandi jaringan Mesh
7 #define MESH_PORT 5656 // Port komunikasi TCP Mesh
8
9 // Konfigurasi pin dan tipe sensor DHT
10 #define DHTPIN 2 // Pin D4 (GPIO 2)
11 #define DHTTYPE DHT22 // Ganti DHT22 jika menggunakan varian DHT22
12 DHT dht(DHTPIN, DHTTYPE);
13
14 // Identitas unik node untuk mempermudah identifikasi teks
15 const char* nodeName = "Node-1";
16
17 Scheduler userScheduler;
18 painlessMesh mesh;
19
20 // Prototipe fungsi pengiriman pesan berkala
21 void sendMessage();
22 Task taskSendMessage(TASK_SECOND * 3, TASK_FOREVER, &sendMessage);
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

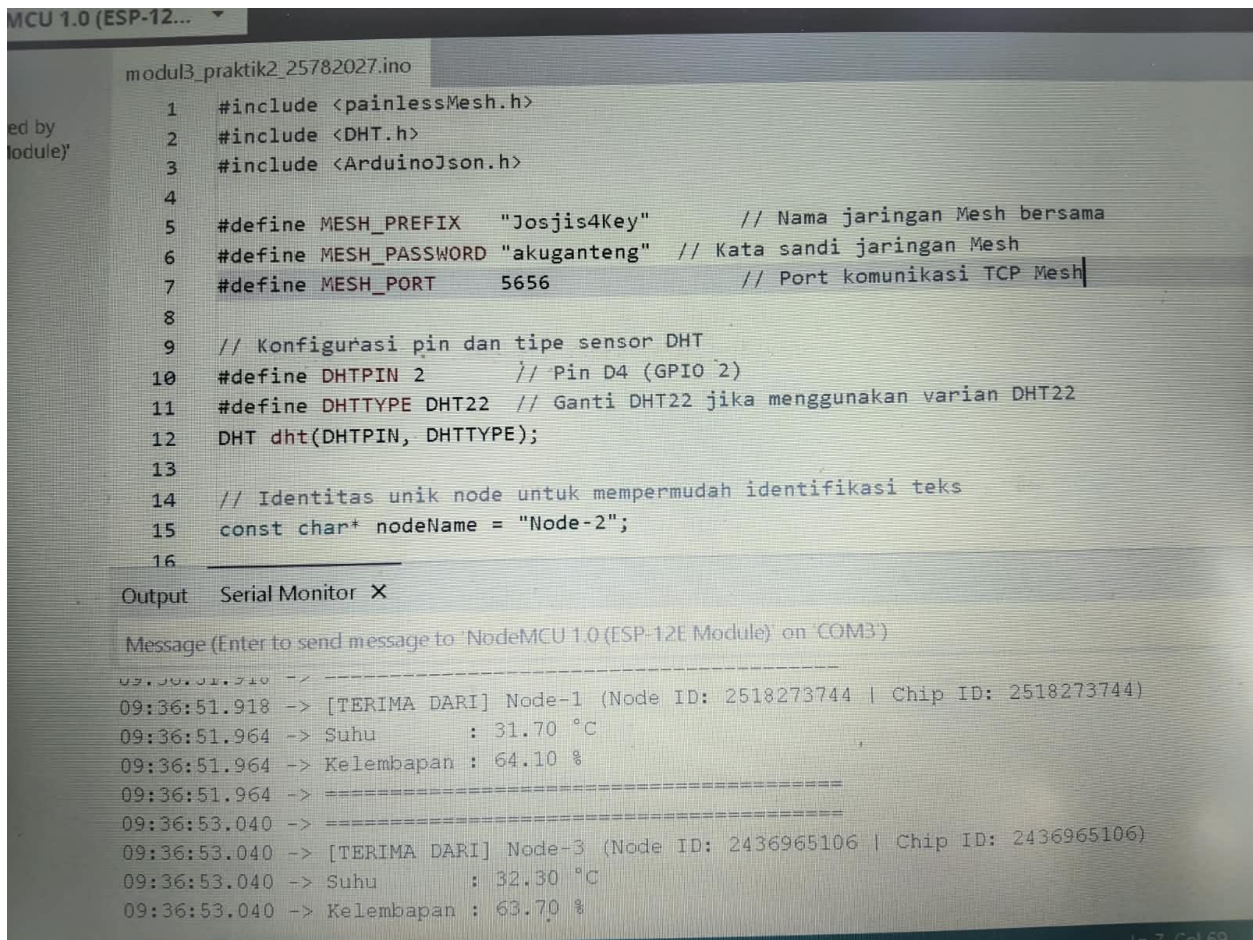
[KIRIM MESH] {"node":"Node-1","chipId":2518273744,"suhu":31.6,"kelembapan":64.2}

[TERIMA DARI] Node-3 (Node ID: 2436965106 | Chip ID: 2436965106)

Suhu : 32.40 °C

Kelembapan : 64.00 %

NODE 2 :



The screenshot shows the Arduino IDE interface with the file 'modul3_praktik2_25782027.ino' open. The code defines a Node-2 with a unique ID, a password, and a port. It includes a DHT sensor and a scheduler. The Serial Monitor shows the output of the code, including the node's identity and sensor data.

```
1 #include <painlessMesh.h>
2 #include <DHT.h>
3 #include <ArduinoJson.h>
4
5 #define MESH_PREFIX "Josjis4Key" // Nama jaringan Mesh bersama
6 #define MESH_PASSWORD "akuganteng" // Kata sandi jaringan Mesh
7 #define MESH_PORT 5656 // Port komunikasi TCP Mesh
8
9 // Konfigurasi pin dan tipe sensor DHT
10 #define DHTPIN 2 // Pin D4 (GPIO 2)
11 #define DHTTYPE DHT22 // Ganti DHT22 jika menggunakan varian DHT22
12 DHT dht(DHTPIN, DHTTYPE);
13
14 // Identitas unik node untuk mempermudah identifikasi teks
15 const char* nodeName = "Node-2";
16
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

09:36:51.910 -> [TERIMA DARI] Node-1 (Node ID: 2518273744 | Chip ID: 2518273744)

09:36:51.964 -> Suhu : 31.70 °C

09:36:51.964 -> Kelembapan : 64.10 %

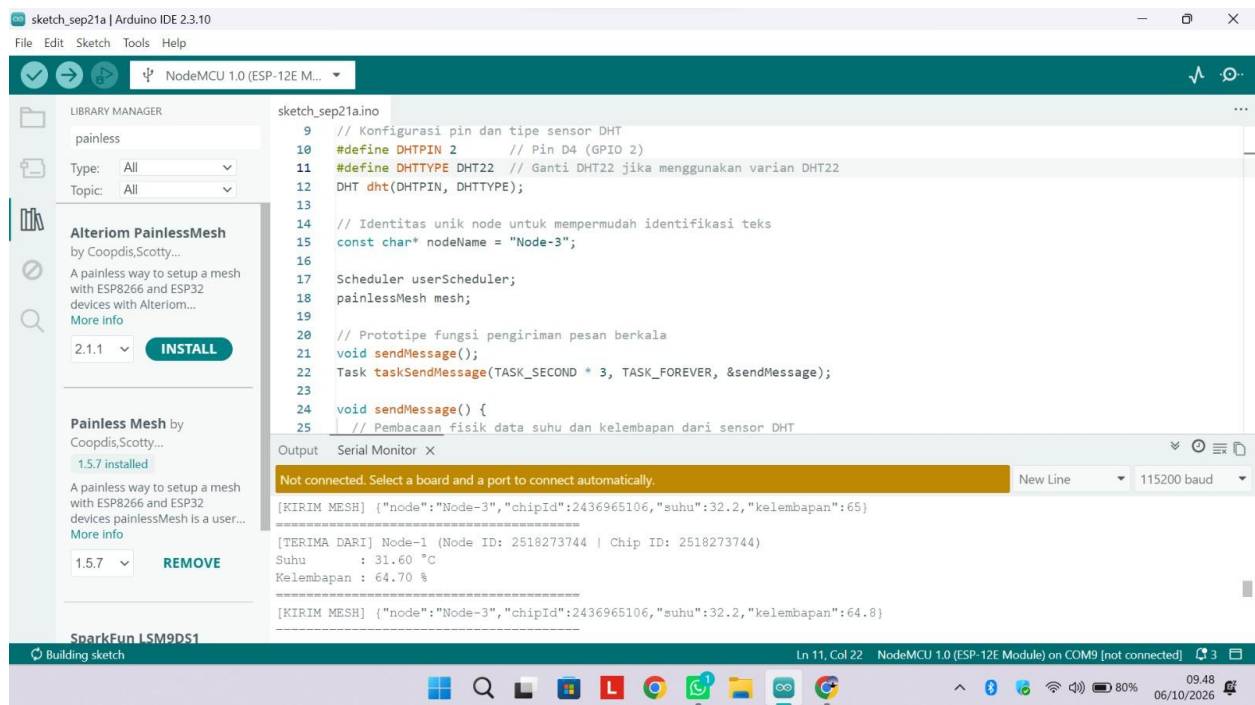
09:36:51.964 -> =====

09:36:53.040 -> [TERIMA DARI] Node-3 (Node ID: 2436965106 | Chip ID: 2436965106)

09:36:53.040 -> Suhu : 32.30 °C

09:36:53.040 -> Kelembapan : 63.70 %

NODE 3 :



RANGKAIAN :

